Methods to Adjust for Dependent Censoring using Inverse Probability of Censoring Weighting

Rune H B Christensen

11 October 2025 – version 1.0

Contents


```
## Warning: pakke 'data.table' blev bygget under R version 4.5.2
```

This note illustrates methods to adjust for censoring on estimation of survival using a small data example with just 6 individuals. We first illustrate adjusting for censoring assuming that the censoring process is independent of covariates, next, we illustrate how to adjust for dependent censoring assuming that the censoring process depends on a covariate. In particular we illustrate the method using a Cox model for the censoring process or using a logistic regression to model the censoring process.

A small data example

In this note we consider a small data example with information on the time to event and time to censoring of six individuals adopted from (Willems et al., 2018):

```
d <- data.table(
  id    = 1:6,
  time  = c(18, 23, 27, 32, 57, 64),
  event = c(0, 1, 0, 1, 0, 1),
  cens  = c(1, 0, 1, 0, 1, 0),
  z     = c(1, 2, 1, 2, 3, 2))
d[]
```

```
##       id  time event  cens     z
##    <int> <num> <num> <num> <num>
## 1:     1    18     0     1     1
## 2:     2    23     1     0     2
## 3:     3    27     0     1     1
## 4:     4    32     1     0     2
## 5:     5    57     0     1     3
## 6:     6    64     1     0     2
```

The covariate `z` will not be used initially but included in later sections.

Data formats

Survival data are sometimes brought onto a form called *counting process format*. This can be useful in cases of delayed entry, when different time scales are included in the analysis simultaneously, when time-varying covariates are considered, or when different baseline hazards need to be assumed for different time periods in Cox models.

For now we bring the data into the counting process format to illustrate that the information in `d` can be presented in this format too. Note that the format is on a time-by-subject level with one row for each time interval for each subject, while in `d` there simply was one row per subject.

Format data in counting process format; time-subject-level (we ignore `z` for now):

```
dcp <- survSplit(Surv(time, event) ~ id,
                 data = d, cut = d[, time]) |> as.data.table()
dcp[, last_time := fifelse(time == max(time), 1, 0), id]
dcp[, cens := last_time - event]
dcp[]
```

```
##        id tstart  time event last_time  cens
##     <int>  <num> <num> <num>     <num> <num>
##  1:     1      0    18     0         1     1
##  2:     2      0    18     0         0     0
##  3:     2     18    23     1         1     0
##  4:     3      0    18     0         0     0
##  5:     3     18    23     0         0     0
##  6:     3     23    27     0         1     1
##  7:     4      0    18     0         0     0
##  8:     4     18    23     0         0     0
##  9:     4     23    27     0         0     0
## 10:     4     27    32     1         1     0
## 11:     5      0    18     0         0     0
## 12:     5     18    23     0         0     0
## 13:     5     23    27     0         0     0
## 14:     5     27    32     0         0     0
## 15:     5     32    57     0         1     1
## 16:     6      0    18     0         0     0
## 17:     6     18    23     0         0     0
## 18:     6     23    27     0         0     0
## 19:     6     27    32     0         0     0
## 20:     6     32    57     0         0     0
## 21:     6     57    64     1         1     0
##        id tstart  time event last_time  cens
```

We used the unique event or censoring times of `d` as the times of splitting the time axis. The result is a dataset and `id` indicator for subjects and time intervals determined by the start of the interval, `tstart` and the end of the interval `time`. The `event` indicator is produced by `survSplit` and we added a `cens` indicator for censoring.

Another similar format is the discrete time or binned data format. Here we also split the time axis but now we split it into time intervals of constant size. For example, we can split the data using unit length time intervals:

```
by <- 1:d[, max(time)]
dbin <- survSplit(Surv(time, event) ~ id,
                  data = d, cut = by) |> as.data.table()
dbin[, last_time := fifelse(time == max(time), 1, 0), id]
dbin[, cens := last_time - event]
dbin[]
```

```
##         id tstart  time event last_time  cens
##      <int>  <num> <num> <num>     <num> <num>
##   1:     1      0     1     0         0     0
##   2:     1      1     2     0         0     0
##   3:     1      2     3     0         0     0
##   4:     1      3     4     0         0     0
##   5:     1      4     5     0         0     0
##  ---
## 217:     6     59    60     0         0     0
## 218:     6     60    61     0         0     0
## 219:     6     61    62     0         0     0
## 220:     6     62    63     0         0     0
## 221:     6     63    64     1         1     0
```

We now proceed to collapse the counting process data to a time-level data set by summarizing the information over individuals. Incidentally this dataset is the same as the subject-level dataset (`d`), but this is may not be the case in real datasets, for instance, because some subjects may experience events at the same time points:

```
dt <- dcp[, .(nrisk = .N, event = sum(event), cens = sum(cens)), keyby=time]
dt0 <- copy(dt)
dt[]
```

```
## Key: <time>
##     time nrisk event  cens
##    <num> <int> <num> <num>
## 1:    18     6     0     1
## 2:    23     5     1     0
## 3:    27     4     0     1
## 4:    32     3     1     0
## 5:    57     2     0     1
## 6:    64     1     1     0
```

Note, however, that we added the `nrisk` variable; the number of subjects at risk at the start of each time interval. Also note that the binned dataset could be similarly collapsed.

Computing survival

Conventional Kaplan-Meier

The conventional non-parametric hazard and Kaplan-Meier (KM) survival estimator is

$$\hat{S}_{km}(t) = \hat{P}[T > t] = 1 - \prod_{i:t_i \leq t} \left(1 - \frac{y_i}{r_i}\right)$$

where

- $h_{kmi} = y_i / r_i$ is the KM nonparametric hazard estimate dividing the event indicator y_i with the number at risk r_i, and
- $\hat{S}_{km}(t)$ is the KM survival estimate.

Compute conventional non-parametric hazard and Kaplan-Meier (KM) survival:

```
dt[, KM_hazard := event / nrisk]
dt[, KM_surv := cumprod(1 - KM_hazard)]
print(dt, digits=3)
```

```
## Key: <time>
##     time nrisk event  cens KM_hazard KM_surv
##    <num> <int> <num> <num>     <num>   <num>
## 1:    18     6     0     1     0.000   1.000
## 2:    23     5     1     0     0.200   0.800
## 3:    27     4     0     1     0.000   0.800
## 4:    32     3     1     0     0.333   0.533
## 5:    57     2     0     1     0.000   0.533
## 6:    64     1     1     0     1.000   0.000
```

IP censoring weighted eCDF

An inverse probability of censoring weighted (IPCW) empirical cumulative distribution function (eCDF) estimator of the survival function is given by

$$S_{ipwc}(t) = 1 - \frac{1}{N} \sum_{i:t_i \leq t} \frac{y_i}{P[C_i \geq t]}$$

where $P[C_i \geq t] = 1 - P[C_i < t]$ is the probability of remaining uncensored up to time t.

Note that using the KM directly on the censoring process would give $P[C_i \leq t]$ which includes time t while what we need is $P[C_i < t]$ not including time t. However, computing $P[C_i \leq t]$ and shifting it one step forward in time gives $P[C_i < t]$:

```
dt <- copy(dt0)
dt[, haz_cens1 := cens / (nrisk - event)]
dt[, haz_cens := c(0, haz_cens1[-.N])]
dt[, P_uncens := cumprod(1 - haz_cens)]
```

The final step is simply to compute the weighted eCDF:

```
dt[, WCDF_surv := 1 - (1/sum(event + cens)) * cumsum(event / P_uncens)]
print(dt, digits=3)
```

```
## Key: <time>
##     time nrisk event  cens haz_cens1 haz_cens P_uncens WCDF_surv
##    <num> <int> <num> <num>     <num>    <num>    <num>     <num>
## 1:    18     6     0     1     0.167    0.000    1.000     1.000
## 2:    23     5     1     0     0.000    0.167    0.833     0.800
## 3:    27     4     0     1     0.250    0.000    0.833     0.800
## 4:    32     3     1     0     0.000    0.250    0.625     0.533
## 5:    57     2     0     1     0.500    0.000    0.625     0.533
## 6:    64     1     1     0       NaN    0.500    0.312     0.000
```

This estimator gives the same survival as the KM estimator and the two estimators can in fact be proven to be identical (Satton and Datta, 2001).

Weighted Kaplan-Meier

The weighted KM estimator

$$\hat{S}_{wkm}(t) = 1 - \prod_{i:t_i \leq t} \left(1 - \frac{y_i \cdot w_i}{r_i \cdot w_i}\right)$$

trivially multiplies the event indicator, y_i and the number at risk r_i by a weight w_i. In this simple case the weights cancel out leading to the same hazard and survival estimator, however, the estimator has more relevance when applied to data in counting process format as we shall see later.

Compute the weighted KM estimator using the inverse probability of remaining uncensored as weights:

```
dt[, W := 1 / P_uncens]
dt[, W_hazard := (event * W) / (nrisk * W)]
dt[, W_surv := cumprod(1 - W_hazard)]
print(dt[, -"haz_cens1"], digits=3)
```

```
## Key: <time>
##     time nrisk event  cens haz_cens P_uncens WCDF_surv     W W_hazard W_surv
##    <num> <int> <num> <num>    <num>    <num>     <num> <num>    <num>  <num>
## 1:    18     6     0     1    0.000    1.000     1.000   1.0    0.000  1.000
## 2:    23     5     1     0    0.167    0.833     0.800   1.2    0.200  0.800
## 3:    27     4     0     1    0.000    0.833     0.800   1.2    0.000  0.800
## 4:    32     3     1     0    0.250    0.625     0.533   1.6    0.333  0.533
## 5:    57     2     0     1    0.000    0.625     0.533   1.6    0.000  0.533
## 6:    64     1     1     0    0.500    0.312     0.000   3.2    1.000  0.000
```

Computation of hazard

To generalize the computation of the hazard of censoring we now show how it can be obtained using a logistic regression model. To obtain the probability of remaining uncensored at the *start* of each interval, we predict the hazard of censoring on the start-time even though we fit the model on the event time (end of interval):

```
dt <- copy(dt0)
dt[, tstart := c(0, time[-.N])]
dt[, f_time := factor(time)] # set f_time to end of interval
lr <- glm(cbind(cens, nrisk - cens) ~ f_time, data=dt, family=binomial)
dt[, f_time := factor(tstart)] # change f_time to start of interval
# Now predict:
dt[tstart > 0 , haz_cens :=
     round(predict(lr, newdata=dt[tstart > 0], type="response"), 8)]
# The hazard of censoring is always 0 at time 0:
dt[tstart == 0, haz_cens := 0]
dt[, P_uncens := cumprod(1 - haz_cens)]
print(dt, digits=3)
```

```
## Key: <time>
## Index: <tstart>
##     time nrisk event  cens tstart f_time haz_cens P_uncens
##    <num> <int> <num> <num>  <num> <fctr>    <num>    <num>
## 1:    18     6     0     1      0      0    0.000    1.000
## 2:    23     5     1     0     18     18    0.167    0.833
## 3:    27     4     0     1     23     23    0.000    0.833
## 4:    32     3     1     0     27     27    0.250    0.625
## 5:    57     2     0     1     32     32    0.000    0.625
## 6:    64     1     1     0     57     57    0.500    0.312
```

Observe that the hazard and estimated probability of remaining uncensored is the same as before at all time points.

Working on the counting process data

Recall that the data in counting process format are:

```
dcp
```

```
##        id tstart  time event last_time  cens
##     <int>  <num> <num> <num>     <num> <num>
##  1:     1      0    18     0         1     1
##  2:     2      0    18     0         0     0
##  3:     2     18    23     1         1     0
##  4:     3      0    18     0         0     0
##  5:     3     18    23     0         0     0
##  6:     3     23    27     0         1     1
##  7:     4      0    18     0         0     0
##  8:     4     18    23     0         0     0
##  9:     4     23    27     0         0     0
## 10:     4     27    32     1         1     0
## 11:     5      0    18     0         0     0
## 12:     5     18    23     0         0     0
## 13:     5     23    27     0         0     0
## 14:     5     27    32     0         0     0
## 15:     5     32    57     0         1     1
## 16:     6      0    18     0         0     0
```

```
## 17:     6     18    23     0         0     0
## 18:     6     23    27     0         0     0
## 19:     6     27    32     0         0     0
## 20:     6     32    57     0         0     0
## 21:     6     57    64     1         1     0
##        id tstart  time event last_time  cens
```

Inverse probability of censoring weights

We let $W_j(t_i) = 1/P[C_{ij} \geq t_i]$ denote the inverse of the probability that the j'th individual remains uncensored up time t_i.

First, estimate the hazard of censoring using a logistic regression model on the counting process data, next compute the probability of remaining uncensored at the start of each interval, and finally compute the inverse probability of censoring weights.

```
dcp[, f_time := factor(time)] # set f_time to end of interval
lr <- glm(cens ~ f_time, data=dcp, family=binomial)
dcp[, f_time := factor(tstart)] # change f_time to start of interval
# Now predict:
dcp[tstart > 0 , haz_cens :=
     round(predict(lr, newdata=dcp[tstart > 0], type="response"), 8)]
# The hazard of censoring is always 0 at time 0:
dcp[tstart == 0, haz_cens := 0]
dcp[, P_uncens := cumprod(1 - haz_cens), id]
dcp[, W := 1 / P_uncens]
print(dcp, digits=3)
```

```
## Index: <tstart>
##        id tstart  time event last_time  cens f_time haz_cens P_uncens     W
##     <int>  <num> <num> <num>     <num> <num> <fctr>    <num>    <num> <num>
##  1:     1      0    18     0         1     1      0    0.000    1.000   1.0
##  2:     2      0    18     0         0     0      0    0.000    1.000   1.0
##  3:     2     18    23     1         1     0     18    0.167    0.833   1.2
##  4:     3      0    18     0         0     0      0    0.000    1.000   1.0
##  5:     3     18    23     0         0     0     18    0.167    0.833   1.2
##  6:     3     23    27     0         1     1     23    0.000    0.833   1.2
##  7:     4      0    18     0         0     0      0    0.000    1.000   1.0
##  8:     4     18    23     0         0     0     18    0.167    0.833   1.2
##  9:     4     23    27     0         0     0     23    0.000    0.833   1.2
## 10:     4     27    32     1         1     0     27    0.250    0.625   1.6
## 11:     5      0    18     0         0     0      0    0.000    1.000   1.0
## 12:     5     18    23     0         0     0     18    0.167    0.833   1.2
## 13:     5     23    27     0         0     0     23    0.000    0.833   1.2
## 14:     5     27    32     0         0     0     27    0.250    0.625   1.6
## 15:     5     32    57     0         1     1     32    0.000    0.625   1.6
## 16:     6      0    18     0         0     0      0    0.000    1.000   1.0
## 17:     6     18    23     0         0     0     18    0.167    0.833   1.2
## 18:     6     23    27     0         0     0     23    0.000    0.833   1.2
## 19:     6     27    32     0         0     0     27    0.250    0.625   1.6
## 20:     6     32    57     0         0     0     32    0.000    0.625   1.6
## 21:     6     57    64     1         1     0     57    0.500    0.312   3.2
##        id tstart  time event last_time  cens f_time haz_cens P_uncens     W
```

Weighted Kaplan-Meier estimator

The weighted KM survival estimator for the counting process data is

$$\hat{S}_{wkm}(t) = \prod_{i:t_i \leq t} \left[1 - \frac{\sum_j W_j(t_i) \cdot y_j(t_i)}{\sum_j W_j(t_i)} \right]$$

The weighted hazard at time t_i, ie. the fraction within the brackets, is then sum of weights for subjects with event divided by the sum of weights for subjects at risk at time t_i

We collapse the data over individuals to get a time-level dataset while computing at each time t_i the weighted hazard:

```
dt2 <- dcp[, .(nrisk = .N, event=sum(event), cens=sum(cens), W = sum(W),
        W_hazard = sum(W[event == 1]) / sum(W)), keyby=time]
## Equivalent computations:
# dcp[, .(nrisk = .N, event=sum(event), cens=sum(cens), W = sum(W),
#         W_hazard = sum(W*event) / sum(W)), time]
# dcp[, .(nrisk = .N, event=sum(event), cens=sum(cens), W = sum(W),
#         W_hazard = weighted.mean(event, W)), time]
# print(dt2, digits=3)
```

Secondly we compute the survival estimate using the weighted hazard:

```
dt2[, W_surv := cumprod(1 - W_hazard)]
print(dt2, digits=3)
```

```
## Key: <time>
##     time nrisk event  cens     W W_hazard W_surv
##    <num> <int> <num> <num> <num>    <num>  <num>
## 1:    18     6     0     1   6.0    0.000  1.000
## 2:    23     5     1     0   6.0    0.200  0.800
## 3:    27     4     0     1   4.8    0.000  0.800
## 4:    32     3     1     0   4.8    0.333  0.533
## 5:    57     2     0     1   3.2    0.000  0.533
## 6:    64     1     1     0   3.2    1.000  0.000
```

and again we recover the same estimates of hazard and survival at all time points.

Hazard estimation using weighted logistic regression

Instead of collapsing the counting process data over time, we can fit a weighted logistic regression using the counting process data for events using the IP censoring weights and then predict the hazard on a small time-level dataset:

```
dcp[, f_time := factor(time)]
lr <- glm(event ~ f_time, data=dcp, family=binomial, weights = W)
pdata <- expand.grid(f_time = dcp[, sort(unique(f_time))]) |>
  as.data.table()
pdata[, lrW_hazard := round(predict(lr, newdata=pdata, type="response"), 8)]
pdata[, lrW_surv := cumprod(1 - lrW_hazard)]
print(pdata, digits=3)
```

```
##    f_time lrW_hazard lrW_surv
##    <fctr>      <num>    <num>
## 1:     18      0.000    1.000
## 2:     23      0.200    0.800
## 3:     27      0.000    0.800
## 4:     32      0.333    0.533
## 5:     57      0.000    0.533
## 6:     64      1.000    0.000
```

Again the hazard and survival estimates agree with previous estimates.

Adjusting for dependent censoring using Cox models

The counting process data format is useful when the methods of interest only use the ranking of the data rather than the value of the event and censoring times themselves. Such methods include the KM estimator and the Cox proportional hazards model. In this section we illustrate how Cox models can be used to adjust for dependent censoring.

We now include the covariate `z` in the counting process format of the data and assume that the censoring process depends on it.

```
dcp2 <- survSplit(Surv(time, event) ~ id + z,
                  data = d, cut = d[, time]) |> as.data.table()
dcp2[, last_time := fifelse(time == max(time), 1, 0), id]
dcp2[, cens := last_time - event]
dcp2[, f_time := factor(time)]
dcp2[]
```

```
##        id     z tstart  time event last_time  cens f_time
##     <int> <num>  <num> <num> <num>     <num> <num> <fctr>
##  1:     1     1      0    18     0         1     1     18
##  2:     2     2      0    18     0         0     0     18
##  3:     2     2     18    23     1         1     0     23
##  4:     3     1      0    18     0         0     0     18
##  5:     3     1     18    23     0         0     0     23
##  6:     3     1     23    27     0         1     1     27
##  7:     4     2      0    18     0         0     0     18
##  8:     4     2     18    23     0         0     0     23
##  9:     4     2     23    27     0         0     0     27
## 10:     4     2     27    32     1         1     0     32
## 11:     5     3      0    18     0         0     0     18
## 12:     5     3     18    23     0         0     0     23
## 13:     5     3     23    27     0         0     0     27
## 14:     5     3     27    32     0         0     0     32
## 15:     5     3     32    57     0         1     1     57
## 16:     6     2      0    18     0         0     0     18
## 17:     6     2     18    23     0         0     0     23
## 18:     6     2     23    27     0         0     0     27
## 19:     6     2     27    32     0         0     0     32
## 20:     6     2     32    57     0         0     0     57
## 21:     6     2     57    64     1         1     0     64
##        id     z tstart  time event last_time  cens f_time
```

The Cox proportional hazards model

The Cox proportional hazards model is

$$h(t; x) = h_0(t) \exp(\beta^T x)$$

with estimated survival times given by

$$S(t; x) = \exp\left[-H_0(t) \exp(\beta^T x)\right]$$

where

$$H_0(t) = \int_0^t h_0(u)\, \mathrm{d}t$$

is the cumulative baseline hazard.

The baseline hazard is not estimated by the Cox model but after model estimation it can be obtained in a post-processing step utilizing the estimated parameters of the model. An often cited estimator is the Breslow estimator:

$$\hat{H}(t) = \sum_{i:t_i \leq t} \frac{d_i}{\sum_{i:t_i \geq t} \exp(\hat{\beta}^T x_i)}$$

but there are other estimators available that primarily differ in how tied event-times are handled. To obtain what is commonly understood as *the* baseline hazard ($H_0(t)$) one would have to set all x_i to zero. However, because $\exp[\beta^T(x_i - z_i)] \log S(t; z_i) = \log S(t; x_i)$ any value of x_i can provide a reference.

In the **survival** package in **R** the `survfit` method for Cox model objects (class `coxph`) provides the estimation of survival. The function has many options, see `help(survfit.coxph)`. The function has a `newdata` argument allowing the specification of covariate values such as those of `z`. The function returns the entire survival curve for all times at which an event occurs for *each* covariate value, ie., one survival curve for each row of `newdata`. Because we only need one survival probability for each row of the `dcp2` dataset we have to extract the survival probability for the relevant time point for each covariate value.

Inverse probability of censoring weights

To estimate the probability of censoring we fit a Cox model with censoring as outcome and `z` as a covariate:

```
fm_cens <- coxph(Surv(time = tstart, time2 = time, event = cens) ~ z,
                 data = dcp2)
summary(fm_cens)
```

```
## Call:
## coxph(formula = Surv(time = tstart, time2 = time, event = cens) ~
##     z, data = dcp2)
##
##   n= 21, number of events= 3
##
##       coef exp(coef) se(coef)      z Pr(>|z|)
## z -1.2702    0.2808   1.1232 -1.131    0.258
##
##   exp(coef) exp(-coef) lower .95 upper .95
## z    0.2808      3.562   0.03106     2.538
##
## Concordance= 0.833  (se = 0.159 )
## Likelihood ratio test= 1.57  on 1 df,   p=0.2
## Wald test            = 1.28  on 1 df,   p=0.3
## Score (logrank) test = 1.45  on 1 df,   p=0.2
```

We loop over each row of the counting process dataset `dcp2` and use `survfit` to estimate the survival curve for each value of `z`. Then we use the `summary` method for `survfit` objects to get the survival probability for a specific time. Here we want the probability of remaining uncensored at the *start* of each time interval so we specify `tstart` in the `times`-argument of `summary`. The probabilities are all saved to a variable `P_uncens` in the `dcp2` dataset:

```
dcp2$P_uncens <- sapply(1:dcp2[, .N], function(i) {
  sf_fm_cens <- survfit(fm_cens, newdata = dcp2[i], stype = 2)
  summary(sf_fm_cens, times = dcp2[i, tstart])$surv
})
```

Notes:

- Because the survival curve of subjects with the same value of `z` is the same we didn't need to fit a survival curve for each row of `dcp2`, however, in settings with continuous covariates often subjects will have a unique covariate vector and it will be needed.
- We used the option `stype = 1` to use a Kaplan-Meier type estimator of survival. The default option (`stype = 2`) exponentiates the cumulative hazard instead and leads to slightly different estimates of `P_uncens` and therefore also different survival estimates.

The IP weights are now simply obtained as the inverse of the estimated probability of remaining uncensored:

```
dcp2[, W := 1 / P_uncens]
print(dcp2, digits=4)
```

```
##        id     z tstart  time event last_time  cens f_time P_uncens     W
##     <int> <num>  <num> <num> <num>     <num> <num> <fctr>    <num> <num>
##  1:     1     1      0    18     0         1     1     18   1.0000 1.000
##  2:     2     2      0    18     0         0     0     18   1.0000 1.000
##  3:     2     2     18    23     1         1     0     23   0.9084 1.101
##  4:     3     1      0    18     0         0     0     18   1.0000 1.000
##  5:     3     1     18    23     0         0     0     23   0.7101 1.408
##  6:     3     1     23    27     0         1     1     27   0.7101 1.408
##  7:     4     2      0    18     0         0     0     18   1.0000 1.000
##  8:     4     2     18    23     0         0     0     23   0.9084 1.101
##  9:     4     2     23    27     0         0     0     27   0.9084 1.101
## 10:     4     2     27    32     1         1     0     32   0.7655 1.306
## 11:     5     3      0    18     0         0     0     18   1.0000 1.000
## 12:     5     3     18    23     0         0     0     23   0.9734 1.027
## 13:     5     3     23    27     0         0     0     27   0.9734 1.027
## 14:     5     3     27    32     0         0     0     32   0.9277 1.078
## 15:     5     3     32    57     0         1     1     57   0.9277 1.078
## 16:     6     2      0    18     0         0     0     18   1.0000 1.000
## 17:     6     2     18    23     0         0     0     23   0.9084 1.101
## 18:     6     2     23    27     0         0     0     27   0.9084 1.101
## 19:     6     2     27    32     0         0     0     32   0.7655 1.306
## 20:     6     2     32    57     0         0     0     57   0.7655 1.306
## 21:     6     2     57    64     1         1     0     64   0.3506 2.852
##        id     z tstart  time event last_time  cens f_time P_uncens     W
```

Survival estimation

Survival estimates adjusted for dependent censoring can now be obtained using the weighted KM estimator

```
dt3 <- dcp2[, .(nrisk = .N, event=sum(event), cens=sum(cens), W = sum(W),
        W_hazard = sum(W[event == 1]) / sum(W)), keyby=time]
dt3[, W_surv := cumprod(1 - W_hazard)]
print(dt3, digits=4)
```

```
## Key: <time>
##     time nrisk event  cens     W W_hazard W_surv
##    <num> <int> <num> <num> <num>    <num>  <num>
## 1:    18     6     0     1 6.000   0.0000 1.0000
## 2:    23     5     1     0 5.738   0.1919 0.8081
## 3:    27     4     0     1 4.637   0.0000 0.8081
## 4:    32     3     1     0 3.691   0.3540 0.5221
## 5:    57     2     0     1 2.384   0.0000 0.5221
## 6:    64     1     1     0 2.852   1.0000 0.0000
```

Of course, the unweighted as well as the weighted KM estimators are already implemented in the **survival** package in **R** and we can utilize this functionality directly to compute the survival probabilities unadjusted for censoring (ie., unweighted, `S_unwt`) and adjusted using the inverse probability of censoring weighting, `S_ipcw`:

```
fm_unwt <- survfit(Surv(tstart, time, event) ~ 1, data=dcp2)
fm_ipcw <- survfit(Surv(tstart, time, event) ~ 1, data=dcp2, weights = W)
dt1 <- with(summary(fm_unwt, times=dcp2[, unique(time)]),
            data.table(time, S_unwt=surv))
dt2 <- with(summary(fm_ipcw, times=dcp[, unique(time)]),
            data.table(time, S_ipcw=surv))
tab <- merge(dt1, dt2, by="time")
print(tab, digits=4)
```

```
## Key: <time>
##     time S_unwt S_ipcw
##    <num>  <num>  <num>
## 1:    18 1.0000 1.0000
## 2:    23 0.8000 0.8081
## 3:    27 0.8000 0.8081
## 4:    32 0.5333 0.5221
## 5:    57 0.5333 0.5221
## 6:    64 0.0000 0.0000
```

These survival estimates replicate those in the Table 3 of (Willems et al., 2018).

The unweighted survival estimates can be understood as an estimate of the survival experience of the study population if censoring had not occurred and censoring were independent of covariates. The IP weighted survival estimates can be understood as an estimate of the survival experience of the study population if censoring had not occurred and censoring depended on the covariate `z`.

Adjusting for dependent censoring using logistic regression

Turning now to modelling of the censoring process using logistic regression models we need the binned dataset with one row for each time interval each subject is at risk:

```
by <- 1:d[, max(time)]
dbin <- survSplit(Surv(time, event) ~ id + z,
                  data = d, cut = by) |> as.data.table()
dbin[, last_time := fifelse(time == max(time), 1, 0), id]
dbin[, cens := last_time - event]
dbin[, f_time := factor(time)]
dbin[]
```

```
##         id     z tstart  time event last_time  cens f_time
##      <int> <num>  <num> <num> <num>     <num> <num> <fctr>
##   1:     1     1      0     1     0         0     0      1
##   2:     1     1      1     2     0         0     0      2
##   3:     1     1      2     3     0         0     0      3
##   4:     1     1      3     4     0         0     0      4
##   5:     1     1      4     5     0         0     0      5
##  ---
## 217:     6     2     59    60     0         0     0     60
## 218:     6     2     60    61     0         0     0     61
## 219:     6     2     61    62     0         0     0     62
## 220:     6     2     62    63     0         0     0     63
## 221:     6     2     63    64     1         1     0     64
```

We cannot use the counting process dataset used in the previous section because the logistic regression model (in the manner that we use it) use the actual value of the times of censoring and event; not only their ranks.

Inverse probability of censoring weights

To estimate the probability of censoring we fit a logistic regression using the binary censoring variable as outcome and `z` as a covariate. As previously, we estimate the (discrete) hazard of censoring by predicting from the logistic regression model:

```
lr <- glm(cens ~ z, data=dbin, family=binomial)
dbin[, haz_cens := round(predict(lr, newdata=dbin, type="response"), 8)]
dbin[tstart == 0, haz_cens := 0]
```

We use the KM estimator to estimate the probability of remaining uncensored at each time point and then compute the inverse probability weights:

```
dbin[, P_uncens := cumprod(1 - haz_cens), id]
dbin[, W := 1 / P_uncens]
print(dbin, digits=3)
```

```
## Index: <tstart>
##         id     z tstart  time event last_time  cens f_time haz_cens P_uncens
##      <int> <num>  <num> <num> <num>     <num> <num> <fctr>    <num>    <num>
##   1:     1     1      0     1     0         0     0      1   0.0000    1.000
##   2:     1     1      1     2     0         0     0      2   0.0286    0.971
##   3:     1     1      2     3     0         0     0      3   0.0286    0.944
##   4:     1     1      3     4     0         0     0      4   0.0286    0.917
##   5:     1     1      4     5     0         0     0      5   0.0286    0.891
##  ---
```

```
## 217:     6     2     59    60     0         0     0     60   0.0120    0.490
## 218:     6     2     60    61     0         0     0     61   0.0120    0.484
## 219:     6     2     61    62     0         0     0     62   0.0120    0.478
## 220:     6     2     62    63     0         0     0     63   0.0120    0.473
## 221:     6     2     63    64     1         1     0     64   0.0120    0.467
##          W
##      <num>
##   1:  1.00
##   2:  1.03
##   3:  1.06
##   4:  1.09
##   5:  1.12
##  ---
## 217:  2.04
## 218:  2.07
## 219:  2.09
## 220:  2.12
## 221:  2.14
```

Because we allowed the logistic regression model for censoring to depend only on `z` and not on time, the fitted values take only as many values as there are unique values of `z` (in addition to the zeros that we assigned at $t = 0$):

```
dbin[, .N, .(haz_cens)]
```

```
##      haz_cens     N
##         <num> <int>
## 1: 0.00000000     6
## 2: 0.02856027    43
## 3: 0.01201324   116
## 4: 0.00500372    56
```

Note, however, that no subjects with a `z` value of 2 are censored leading to an unstable fit strongly dependent on the implicit linearity assumption:

```
dbin[, .(cens = sum(cens)), z]
```

```
##        z  cens
##    <num> <num>
## 1:     1     2
## 2:     2     0
## 3:     3     1
```

Survival estimation

To obtain survival probabilities adjusted for censoring we fit a weighted logistic regression using the IP weights but now with *event* as outcome. We then predict to a small dataset and compute the standard KM estimate of survival:

```
lr <- glm(event ~ f_time, data=dbin, family=binomial, weights = W)
pdata <- expand.grid(time = by) |> as.data.table()
pdata[, f_time := factor(time)]
pdata[, lrW_hazard := round(predict(lr, newdata=pdata[time > 0], type="response"), 8)]
pdata[, lrW_surv := cumprod(1 - lrW_hazard)]
## In this case the same as the following:
# pdata <- expand.grid(time = d[, time]) |> as.data.table()
# pdata[, f_time := factor(time)]
# pdata[, lrW_hazard := round(predict(lr, newdata=pdata, type="response"), 8)]
# pdata[, lrW_surv := cumprod(1 - lrW_hazard)]
print(pdata[time %in% d[, time]], digits=3)
```

```
##     time f_time lrW_hazard lrW_surv
##    <int> <fctr>      <num>    <num>
## 1:    18     18      0.000    1.000
## 2:    23     23      0.188    0.812
## 3:    27     27      0.000    0.812
## 4:    32     32      0.357    0.522
## 5:    57     57      0.000    0.522
## 6:    64     64      1.000    0.000
```

To 2 decimals we obtain the same censoring-adjusted survival estimates as we did using the Cox model, but the difference to the unadjusted survival estimates are also small. The results of both methods would also change with other settings and modeling assumptions.

Effect of functional form in censoring model

The inverse probability weights and consequently the resulting survival estimates depend on the specification of the covariate `z` and time, and how they may interact in the logistic regression model for censoring. The following table summarizes the effect of the functional form in the censoring model on the estimated survival probabilities:

```
by <- 1:d[, max(time)]
dbin <- survSplit(Surv(time, event) ~ id + z,
                  data = d, cut = by) |> as.data.table()
dbin[, time_orig := time]
dbin[, tstart_orig := tstart]
dbin[, last_time := fifelse(time == max(time), 1, 0), id]
dbin[, cens := last_time - event]
# dbin[, f_time := factor(time)]
dbin[, f_z := factor(z)]
# dbin[]

forms <- c("1", "time", "f_time",
           "z", "z + time", "z + poly(time, 2)", "z + f_time",
           "z * time",
           "f_z", "f_z + time", "f_z * time")#, "f_z + f_time")

tab <- lapply(forms, function(form) { # form <- forms[5]
  dbin[, time := time_orig]
  dbin[, f_time := factor(time_orig)]
  lr_c <- glm(formula(paste("cens ~", form)), data=dbin, family=binomial)
```

```
  dbin[, time := tstart_orig]
  dbin[, f_time := factor(tstart_orig)]
  dbin[tstart > 0, haz_cens := round(predict(lr_c, newdata=dbin[tstart > 0],
                                             type="response"), 8)]
  dbin[tstart == 0, haz_cens := 0]
  dbin[, P_uncens := cumprod(1 - haz_cens), id]
  dbin[, W := 1 / P_uncens]

  dbin[, time := time_orig]
  dbin[, f_time := factor(time_orig)]
  lr_e <- glm(event ~ f_time, data=dbin, family=binomial, weights = W)
  pdata <- expand.grid(time = d[, time]) |> as.data.table()
  pdata[, f_time := factor(time)]
  pdata[, lrW_hazard := round(predict(lr_e, newdata=pdata, type="response"), 8)]
  pdata[, lrW_surv := cumprod(1 - lrW_hazard)]
  pdata[, form := form]
  pdata[]
}) |> rbindlist()
tab[, form := factor(form, levels=forms)]

tab2 <- dcast(tab, time ~ form, value.var = "lrW_surv")
print(tab2, digits=3)
```

```
## Key: <time>
##     time     1  time f_time     z z + time z + poly(time, 2) z + f_time
##    <num> <num> <num>  <num> <num>    <num>             <num>      <num>
## 1:    18 1.000 1.000  1.000 1.000    1.000             1.000      1.000
## 2:    23 0.800 0.800  0.800 0.812    0.818             0.817      0.814
## 3:    27 0.800 0.800  0.800 0.812    0.818             0.817      0.814
## 4:    32 0.533 0.533  0.533 0.522    0.534             0.531      0.522
## 5:    57 0.533 0.533  0.533 0.522    0.534             0.531      0.522
## 6:    64 0.000 0.000  0.000 0.000    0.000             0.000      0.000
##    z * time   f_z f_z + time f_z * time
##       <num> <num>      <num>      <num>
## 1:    1.000 1.000      1.000      1.000
## 2:    0.838 0.861      0.819      0.827
## 3:    0.838 0.861      0.819      0.827
## 4:    0.558 0.630      0.546      0.551
## 5:    0.558 0.630      0.546      0.551
## 6:    0.000 0.000      0.000      0.000
```

Notes:

- If the censoring model does not depend on `z` it does not matter how it depends on time: In all cases the survival estimates unadjusted for censoring are achieved.
- Any censoring model which depends on `z` leads to some alteration of the survival probability and the survival probabilities depend on the particular combination for `z` and `time`.

Remarks on the methods

- Adjusting for dependent censoring can be useful when interest is in accurately estimating the survival curve or the survival probability at a particular point in time. One could estimate the probability of

remaining uncensored under various counterfactual scenarios about the censoring process by specifying particular covariate settings.
- A more common use case is probably that of adjusting for differential dependent censoring between treatment or exposure groups. This could estimate counterfactual survival contrasts such as the survival if all subjects had been uncensored and received one level of exposure as compared to the survival if all subjects had been uncensored and received another level of exposure.

On assumptions of Cox and logistic regression methods:

- The Cox model for the censoring process makes the (potentially strong) assumption of proportional censoring hazards. The logistic regression method does not make this assumption and allows a large degree of flexibility in the shape of the hazard using only a few parameters.
- The Cox model for the censoring process requires a dataset in counting process format while the logistic regression method requires a discrete time dataset. In the case above we chose a small discrete time interval size leading to a dataset that was larger than the Cox dataset. The interval size is necessarily context dependent and can often be chosen such that the final dataset size is either smaller, comparable or larger than that required for the Cox model.
- In some cases the proportionality assumption of the Cox model is a rather severe restriction, though there are methods to overcome it. While the logistic regression is on paper a more parametric approach it actually imposes less strong assumptions on the hazard shapes than the Cox model.

Appendix

R session info:

```
sessionInfo()
```

```
## R version 4.5.0 (2025-04-11 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 22631)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Danish_Denmark.utf8  LC_CTYPE=Danish_Denmark.utf8
## [3] LC_MONETARY=Danish_Denmark.utf8 LC_NUMERIC=C
## [5] LC_TIME=Danish_Denmark.utf8
##
## time zone: Europe/Copenhagen
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] survival_3.8-3    data.table_1.17.8
##
## loaded via a namespace (and not attached):
##  [1] digest_0.6.37     fastmap_1.2.0     xfun_0.52         Matrix_1.7-3
##  [5] lattice_0.22-6    splines_4.5.0     knitr_1.50        htmltools_0.5.8.1
##  [9] rmarkdown_2.29    cli_3.6.5         grid_4.5.0        compiler_4.5.0
```

```
## [13] rprojroot_2.0.4   tools_4.5.0       evaluate_1.0.3    yaml_2.3.10
## [17] rlang_1.1.6
```